

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH MẠNG

**Đề tài: Ironfront Reborn – Trò chơi bắn súng nhiều người chơi
với tầng mạng TCP/UDP tự xây dựng**

Lớp: D23CTPM01-B
Nhóm bài tập lớn: 10
Sinh viên thực hiện: Nguyễn Tự Kiên
Mã sinh viên: B23DCCN465

Báo cáo kiến trúc và giao diện hệ thống

Hà Nội – 2026

Mục lục

I. NỘI DUNG NHÓM	3
1. Mô tả kiến trúc chung của hệ thống	3
1.1. Bài toán và phạm vi	3
1.2. Kiến trúc ba tiến trình	3
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu	5
1.4. Kiến trúc phân lớp mã nguồn	5
1.5. Kiến trúc triển khai	6
1.6. Phân công theo phân hệ	7
2. Mô tả thiết kế chung của hệ thống	7
2.1. Mô hình thẩm quyền	7
2.2. Luồng nghiệp vụ đầu – cuối	8
2.3. Các kỹ thuật netcode cốt lõi	9
2.4. Thiết kế giao thức	9
2.5. Tham số thời gian	10
2.6. Phân loại đối tượng đồng bộ	11
2.7. Bảo mật ở mức phù hợp phạm vi	11
2.8. Các quyết định kiến trúc đã chốt	11
2.9. Quy trình phát triển và bảo đảm chất lượng	11
II. NỘI DUNG CÁ NHÂN	13
3. Biểu đồ và giao diện nội dung cá nhân thực hiện	13
3.1. Kiến trúc thành phần Master Server	13
3.2. Đóng khung tin trên luồng TCP	13
3.3. Luồng dịch vụ sẵn chờ	15
3.4. Vòng đời kết nối và phòng	16
3.5. Mô hình dữ liệu	16
3.6. Giao diện lập trình và giao diện giám sát	17
3.7. Hạ tầng triển khai	18

I. NỘI DUNG NHÓM

1. Mô tả kiến trúc chung của hệ thống

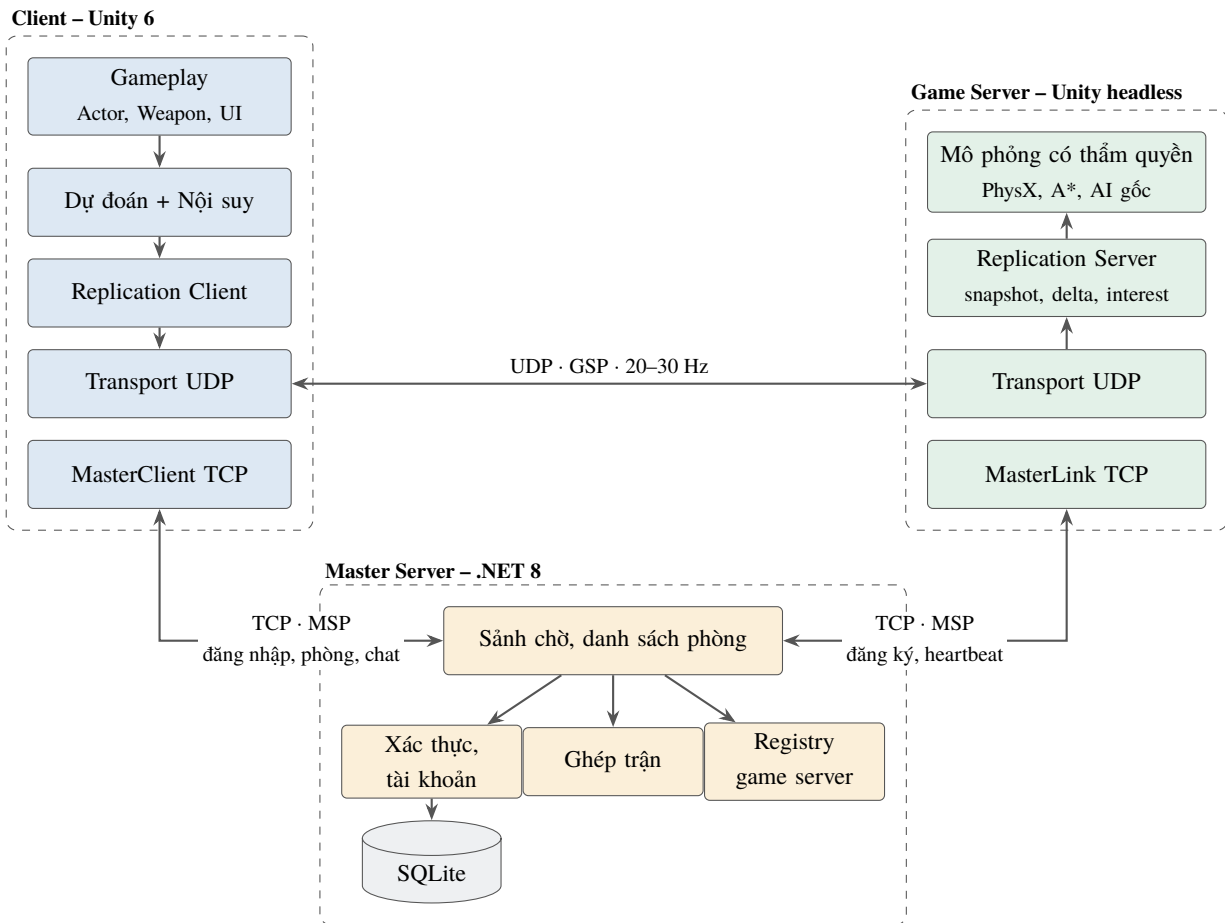
1.1. Bài toán và phạm vi

Ironfront Reborn chuyển mã nguồn trò chơi bắn súng góc nhìn thứ nhất *Ravenfield Beta 5* (Unity 6000.3, vốn chỉ chơi đơn) thành một trò chơi **nhiều người chơi, máy chủ nắm quyền quyết định** (server-authoritative). Yêu cầu cốt lõi của môn Lập trình mạng: **toàn bộ tầng mạng được viết tay trên socket TCP và UDP** thô qua `System.Net.Sockets` — không dùng Mirror, Photon, Netcode for GameObjects, KCP, gRPC hay WebSocket.

Tiêu chí	Giá trị thiết kế
Quy mô một trận	16 người chơi thật + 32 bot AI (tối đa 64 actor, 16 phương tiện)
Tần số mô phỏng / snapshot / gửi input	30 Hz / 20 Hz / 30 Hz
Băng thông xuống mỗi client	≤ 8 KB/s (đo trên server đã triển khai: 1,67 KB/s)
Triển khai	LAN trước, sau đó VPS công cộng (Docker Compose, TLS cho kênh TCP)
Công nghệ	.NET 8 (master server, công cụ), netstandard2.1 (thư viện dùng chung), Unity 6 (client và game server headless)

1.2. Kiến trúc ba tiến trình

Hệ thống gồm **ba loại tiến trình** độc lập, giao tiếp qua **hai giao thức tự định nghĩa**: **MSP** (Master Server Protocol, trên TCP) cho dịch vụ sảnh chờ và **GSP** (Game Server Protocol, trên UDP) cho lưu lượng trong trận. Hai giao thức gặp nhau ở đúng một điểm: **joinTicket** được master ký HMAC và game server tự kiểm tra.



Hình 1.1. Kiến trúc tổng thể ba tiến trình và hai giao thức MSP/GSP

Tiến trình	Nền tảng	Trách nhiệm chính	Giao tiếp
Client	Unity 6, bản build thường	Hiển thị, nhận input, dự đoán chuyển động người chơi cục bộ, nội suy actor ở xa, giao diện sảnh và HUD	TCP tới Master; UDP tới Game Server
Game Server	Unity 6 headless (UNITY_SERVER)	Chạy mô phỏng có thẩm quyền (vật lý, AI, bắn, điểm chiếm), tạo snapshot riêng cho từng client, bù trễ	Nhận UDP từ client; TCP tới Master
Master Server	.NET 8 console, SQLite	Tài khoản, phiên, phòng, ghép trận, chat sảnh, registry game server, cấp joinTicket	Nhận TCP từ client và game server

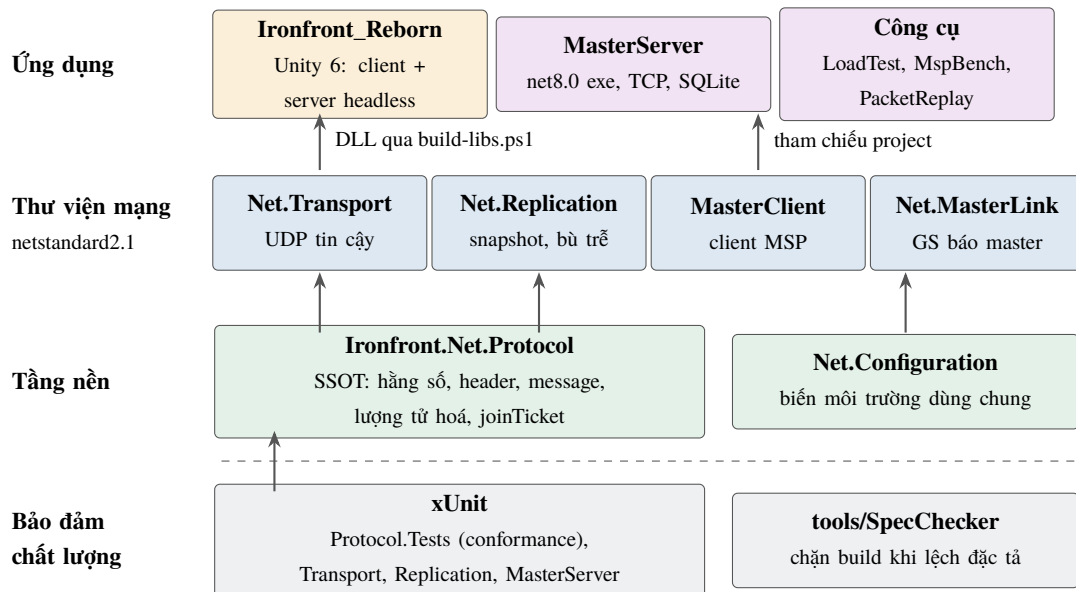
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu

Đặc tính	TCP – Master Server	UDP – Game Server
Dùng cho	Đăng nhập, danh sách phòng, tạo/vào phòng, ghép trận, chat sảnh, heartbeat server	Mọi lưu lượng trong trận: input, snapshot, sự kiện gameplay
Tần suất	Vài gói/phút mỗi client	20–60 gói/giây mỗi chiều
Mất gói	Không chấp nhận (mất LOGIN_RES là treo màn hình)	Chấp nhận với trạng thái: snapshot sau thay thế snapshot trước
Nếu đổi chỗ	Phải tự viết lại độ tin cậy mà TCP đã làm tốt	Head-of-line blocking: một gói mất chặn mọi gói sau; Nagle + delayed ACK thêm 40–200 ms
Đóng khung	Tiền tố độ dài 4 byte + JSON	Datagram tự nhiên, header nhị phân 16 byte

Không dùng WebSocket: WebSocket chạy trên TCP nên thừa hưởng nguyên vẹn head-of-line blocking, cộng thêm chi phí bắt tay HTTP — một ràng buộc chỉ cần cho trình duyệt, không cần cho client desktop.

1.4. Kiến trúc phân lớp mã nguồn

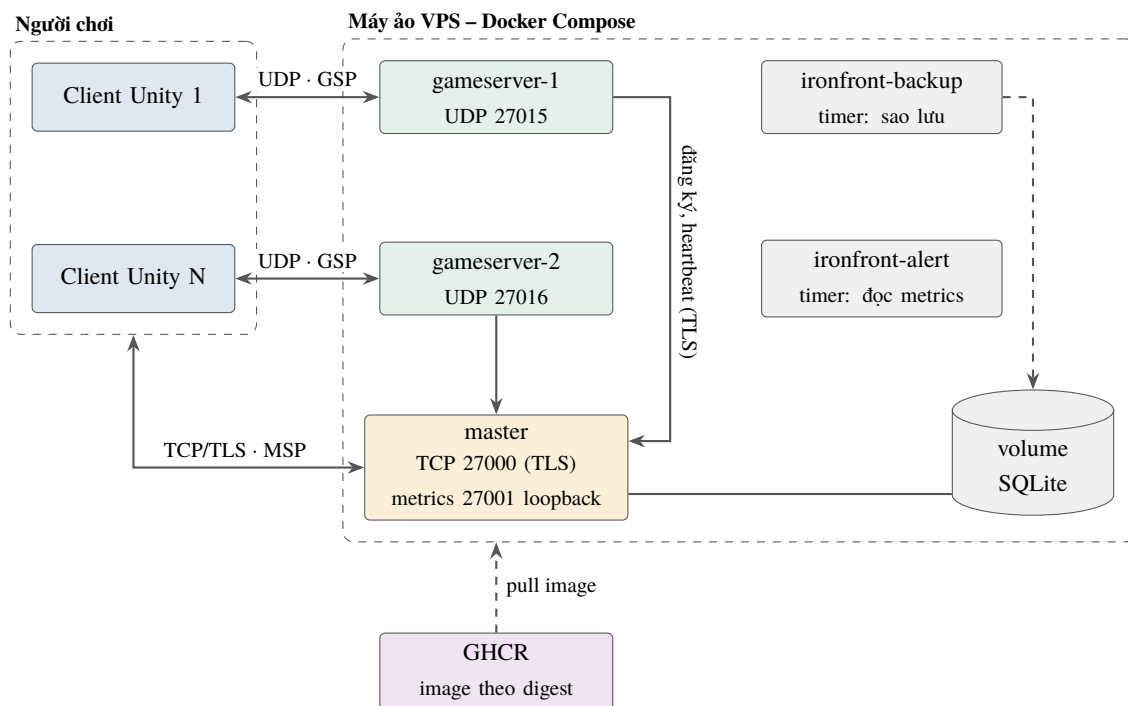
Mã nguồn chia thành các thư viện .NET thuần (netstandard2.1) **không bao giờ tham chiếu** UnityEngine; dự án Unity dùng chúng dưới dạng DLL do tools/build-libs.ps1 sinh vào Assets/Plugins. Nhờ vậy logic mạng được kiểm thử bằng xUnit trong vài giây, không cần mở Unity Editor.



Hình 1.2. Phân lớp thư viện và quan hệ phụ thuộc giữa các project trong solution

Project	Loại	Dòng mã	Vai trò
Ironfront.Net.Protocol	netstandard2.1	5 300	Nguồn sự thật duy nhất của định dạng dây
Ironfront.Net.Transport	netstandard2.1	4 100	Độ tin cậy trên UDP, kênh, phân mảnh, nghẽn, giả lập mạng
Ironfront.Net.Replication	netstandard2.1	18 500	Snapshot, delta, interest, tick server, bù trễ, trận đấu
Ironfront.MasterServer	net8.0 exe	4 300	Framing, dispatch, xác thực, sánh, ghép trận, TLS, metrics
MasterClient, MasterLink	netstandard2.1	1 250	API cho client Unity và liên kết game server – master
Assets/Scripts/Net	Unity C#	14 600	Lớp keo Unity: bootstrap, dự đoán, hiển thị, giao diện

1.5. Kiến trúc triển khai



Hình 1.3. Sơ đồ triển khai trên một máy ảo: một master và hai game server headless

Image được kéo từ GHCR theo digest cố định; bí mật (IRONFRONT_SHARED_SECRET, chứng chỉ TLS) lấy từ tệp .env không commit. Sao lưu và cảnh báo chạy bằng systemd timer *ngoài* container, vì bộ phát hiện sự cố phải nằm ngoài thứ nó giám sát. Hạ tầng còn có cấu hình Terraform (tạo VM, cloud-init) và fly.io làm phương án thay thế.

1.6. Phân công theo phân hệ

Sinh viên	Mã SV	Phân hệ	Phạm vi mã nguồn chính
Nguyễn Thành Trung	B23DCCN861	Client Unity	Assets/Scripts/Net/Client, Net/Input
Nguyễn Minh Toàn	B23DCCN832	Tầng giao vận UDP	Ironfront.Net.Transport
Nguyễn Duy Nghĩa	B23DCCN600	Replication, mô phỏng server	Ironfront.Net.Replication, Net/Server
Nguyễn Tự Kiên	B23DCCN465	Master Server TCP, hạ tầng	Ironfront.MasterServer, MasterClient, infra

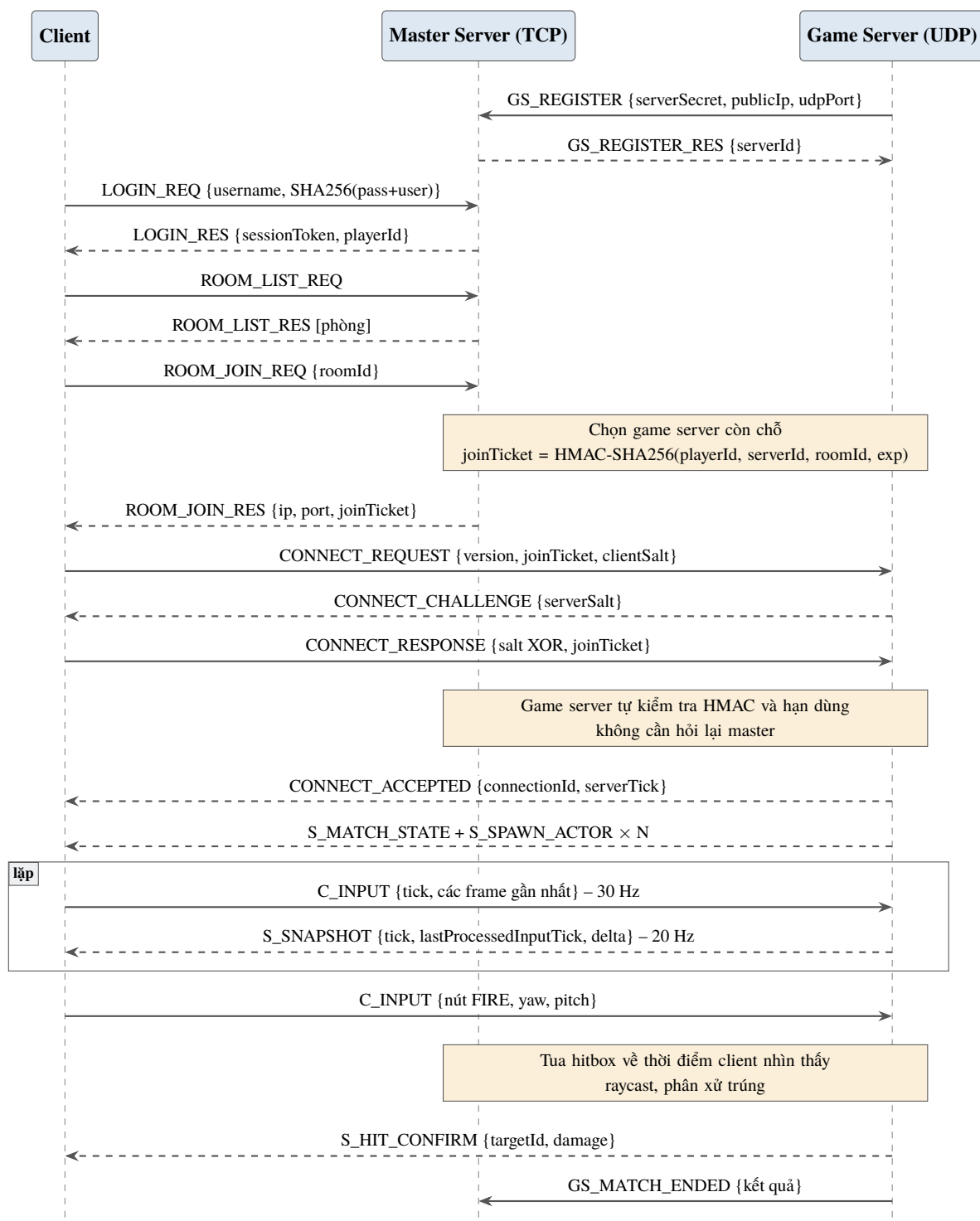
2. Mô tả thiết kế chung của hệ thống

2.1. Mô hình thẩm quyền

Hệ thống **hoàn toàn do server quyết định**: client chỉ gửi *ý định* (input), không bao giờ gửi kết quả. Nhờ vậy gian lận tốc độ, dịch chuyển tức thời, bắn nhanh hay đạn vô hạn trở nên bất khả thi về cấu trúc thay vì chỉ bị phát hiện.

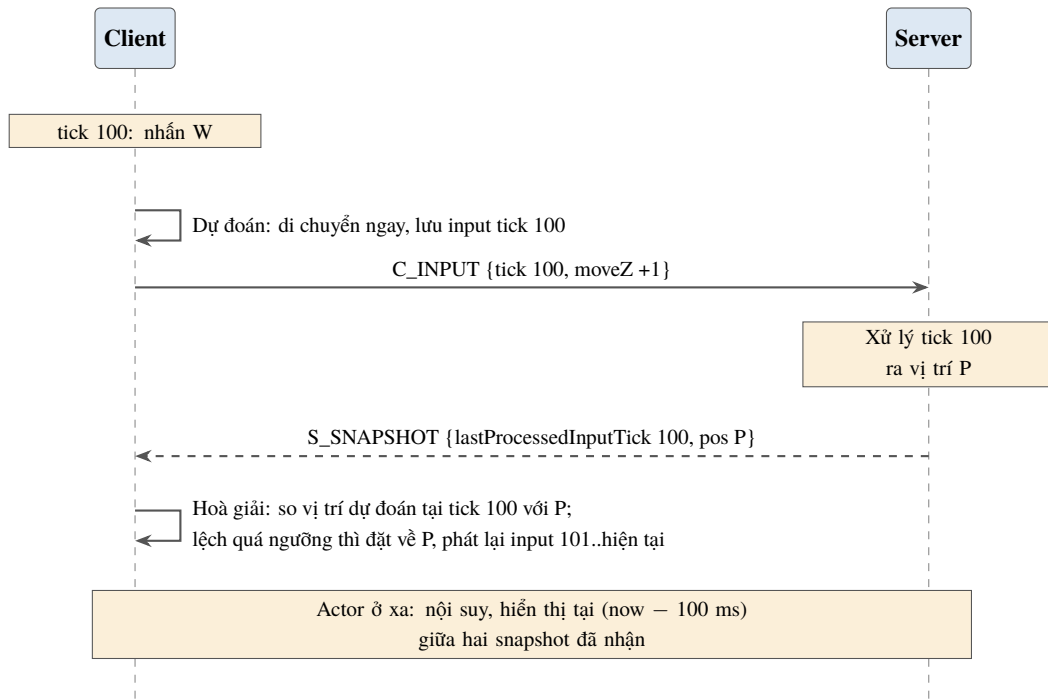
Vấn đề	Ai quyết định	Client được làm gì
Vị trí nhân vật	Server	Dự đoán ngay, sau đó hoà giải với server
Trúng đạn, sát thương	Server	Hiện hiệu ứng dự đoán, chờ S_HIT_CONFIRM
Máu, chết, hồi sinh	Server	Chỉ hiển thị
Độ tản đạn, giật súng	Server	Giật cục bộ cho cảm giác, không ảnh hưởng viên đạn thật
Bot AI, điểm chiếm, điểm số	Server	Nội suy, hiển thị
Lên xuống ghế phương tiện	Server	Gửi yêu cầu C_SEAT_REQUEST
Camera, UI, âm thanh, ragdoll	Client	Toàn quyền, không đồng bộ

2.2. Luồng nghiệp vụ đầu – cuối



Hình 2.1. Luồng đầu – cuối: đăng nhập, vào phòng, kết nối game server, chơi, kết thúc trận

2.3. Các kỹ thuật netcode cốt lõi

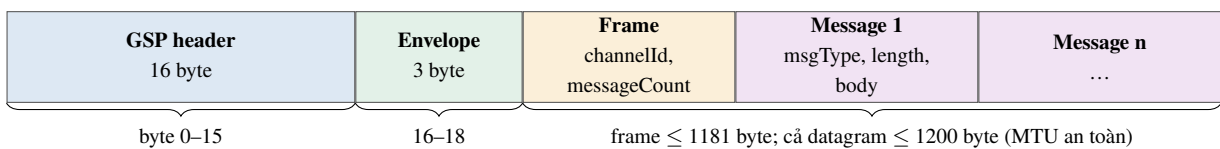


Hình 2.2. Dự đoán phía client, hoà giải với server và nội suy actor ở xa

Kỹ thuật	Mục đích	Tham số
Dự đoán phía client	Người chơi cục bộ di chuyển ngay khi nhấn phím	Cùng MovementSimulation ở client và server
Hoà giải	Sửa sai khi dự đoán lệch, phát lại input chưa xác nhận	Ngưỡng lệch 0,1 m
Nội suy thực thể	Actor ở xa chuyển động mượt dù snapshot chỉ 20 Hz	Bộ đệm 100 ms
Bù trễ	Người ping cao vẫn bắn trúng thứ họ thấy	Tua tối đa 200 ms, lịch sử hitbox 1 s

2.4. Thiết kế giao thức

2.4.1. GSP – cấu trúc một datagram PAYLOAD



Hình 2.3. Ba lớp của một datagram PAYLOAD: header GSP, envelope kênh của transport, frame message của ứng dụng

Kênh	Kiểu	Dữ liệu	Khi mất gói
0	Không tin cậy, không thứ tự	Ping/pong	Bỏ qua
1	Không tin cậy, có thứ tự (server → client)	S_SNAPSHOT, S_WEAPON_FIRE, S_VEHICLE_SNAPSHOT	Bỏ qua; gói cũ bị huỷ
2	Tin cậy, có thứ tự	Sự kiện gameplay, spawn/despawn, chat, C_ACK_BASELINE	Gửi lại tới khi được ack
3	Không tin cậy, có thứ tự (client → server)	C_INPUT, C_VEHICLE_INPUT	Bỏ qua; bù bằng dư thừa

Độ tin cậy áp dụng **theo kênh**, không theo kết nối: một sự kiện bị mất trên kênh 2 không làm nghẽn snapshot trên kênh 1 — ưu điểm cốt lõi so với TCP vốn ép mọi thứ vào một luồng.

2.4.2. MSP – khung tin trên TCP

length u32, big-endian	msgType u16, little-endian	body UTF-8 JSON, tối đa 64 KB
----------------------------------	--------------------------------------	---

Hình 2.4. Khung tin MSP: tiền tố độ dài giải quyết bài toán ranh giới thông điệp trên luồng byte TCP

2.4.3. joinTicket – cầu nối TCP và UDP

playerId u32	serverId u16	roomId u16	expiresAtUnixMs u64 (60 s)	displayName 16 byte UTF-8	hmac HMAC-SHA256, 32 byte
32 byte dữ liệu được ký					32 byte chữ ký

Hình 2.5. Cấu trúc joinTicket 64 byte

Master ký vé bằng SHARED_SECRET; game server kiểm tra chữ ký bằng FixedTimeEquals và kiểm hạn dùng. Không cần gọi lại master, nên **master có thể dừng giữa trận mà trận vẫn tiếp tục**.

2.5. Tham số thời gian

Tham số	Giá trị	Ghi chú
Tick mô phỏng server	30 Hz (33,3 ms)	Theo đồng hồ thực, không theo số lần FixedUpdate
Snapshot	20 Hz (50 ms)	Rẻ hơn 30 Hz, nội suy bù độ mượt
Gửi input	30 Hz	Mỗi gói lặp lại các frame gần nhất
Bộ đệm nội suy	100 ms	Chịu được mất một snapshot
Cửa sổ bù trễ / lịch sử hitbox	200 ms / 1 s	Chặn lạm dụng ping giả
Keep-alive / timeout UDP	1 s / 10 s	
Heartbeat MSP client / game server	15 s	Phát hiện kết nối TCP nửa mở

2.6. Phân loại đối tượng đồng bộ

Loại	Ví dụ	Cách xử lý
Actor được đồng bộ	Người chơi, bot	Có actorId u16, có trong snapshot, mã hoá delta
Phương tiện	Xe, trực thăng, tàu	Luồng riêng S_VEHICLE_SNAPSHOT
Chỉ ở server	Lịch sử hitbox, dữ liệu AI	Không bao giờ gửi
Chỉ ở client	Camera, UI, particle, ragdoll, âm thanh	Không bao giờ gửi
Sự kiện một lần	Chết, nổ, phát súng, chiếm điểm	Kênh tin cậy có thứ tự
Tính	Địa hình, công trình, điểm hồi sinh	Có sẵn trong scene hai phía

2.7. Bảo mật ở mức phù hợp phạm vi

Mối đe dọa	Biện pháp	Tầng
Tăng tốc, dịch chuyển, bắn nhanh, đạn vô hạn	Server chỉ nhận input; kẹp tốc độ; tự quản lý cooldown và đạn	Replication
Mạo danh người chơi	connectionId gắn với playerId trong joinTicket đã ký	Transport
Khuếch đại giả mạo IP	Bắt tay challenge không lưu trạng thái (cookie HMAC)	Transport
Gói rác, lụt gói	Kiểm protocolId; giới hạn tốc độ theo IP; giới hạn nhóm phân mảnh	Transport
Nghe lén mật khẩu	TLS và ghim vân tay chứng chỉ; SHA-256 phía client, bcrypt phía server	Master
Tràn bộ nhớ, Slowloris	Khung tối đa 64 KB; hạn 30 s trước xác thực; 5 kết nối/IP	Master
SQL injection, dò mật khẩu	Truy vấn tham số hoá; 5 lần/phút/IP; khoá 15 phút sau 10 lần sai	Master

2.8. Các quyết định kiến trúc đã chốt

Mã	Quyết định	Lý do
AD-1	Server nắm quyền, không có listen-server	Thẩm quyền rõ ràng; tránh sửa 21 singleton
AD-2	Game server là Unity headless	Tái dùng PhysX, A*, AI; tiết kiệm 8–12 tuần
AD-3	Không theo đuổi tất định (lockstep)	Random rải rác 27 tệp; PhysX không tất định
AD-4	Ragdoll chỉ là hiệu ứng cục bộ	Tiết kiệm khoảng 1,7 MB/s
AD-5	Transport, Replication là thư viện .NET thuần	Kiểm thử bằng xUnit, tránh xung đột Unity
AD-7	Snapshot không tin cậy, sự kiện tin cậy	Đúng bản chất trạng thái và sự kiện
AD-8	Không dùng WebSocket	Yêu cầu môn học và head-of-line blocking

2.9. Quy trình phát triển và bảo đảm chất lượng

- **Đặc tả giao thức đóng băng** và `ProtocolConstants.cs` là nguồn sự thật duy nhất; `tools/SpecChecker` làm hỏng build nếu hai bên lệch nhau.
- **Tách người viết và người kiểm:** phân hệ Transport viết bit-packing, phân hệ Replication viết bộ test conformance bằng hex viết tay theo đặc tả.
- **CI** chạy trên Ubuntu và Windows: build coi cảnh báo là lỗi, chạy test, kiểm tra lệch đặc tả.

- **Definition of Done:** đã chạy và xem đầu ra, test phân hệ xanh, toàn bộ test xanh, Unity biên dịch sạch, đã merge và viết báo cáo phase.

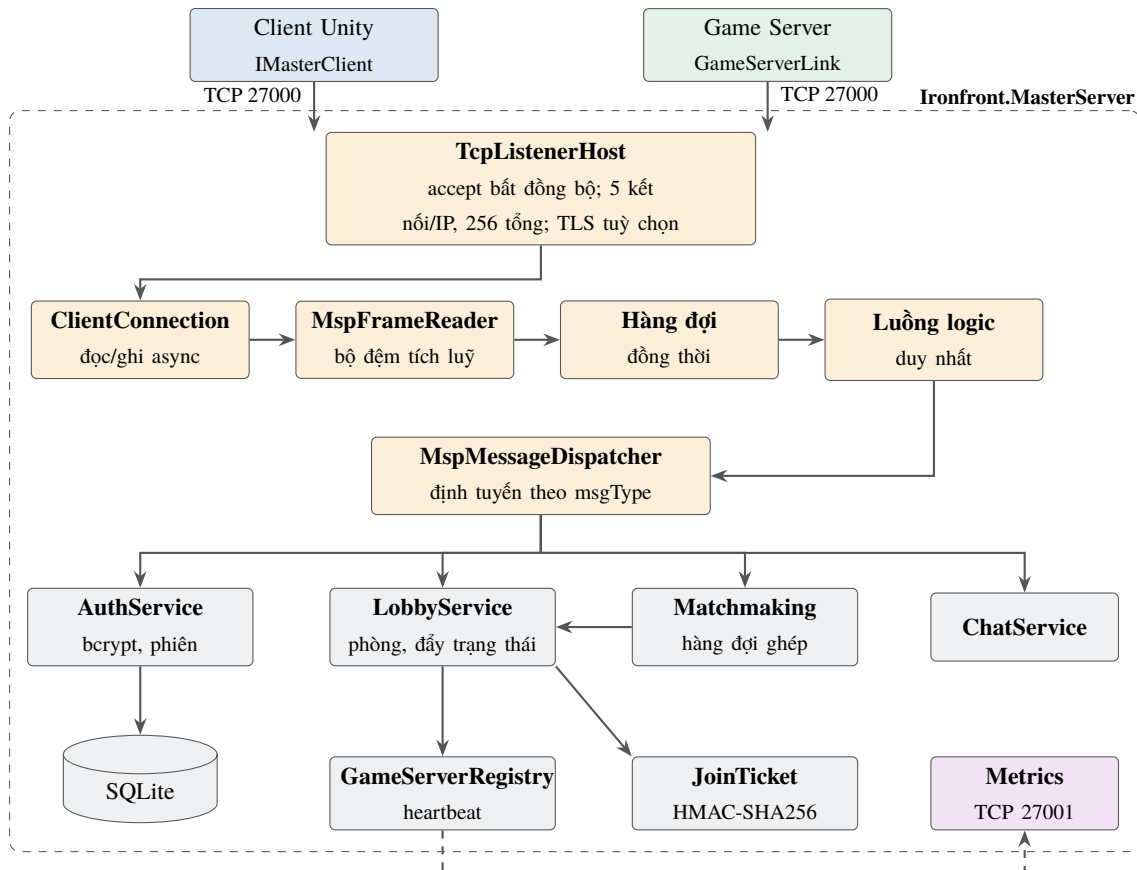
II. NỘI DUNG CÁ NHÂN

Sinh viên thực hiện: Nguyễn Tự Kiên (B23DCCN465) – phân hệ Master Server TCP và hạ tầng.

3. Biểu đồ và giao diện nội dung cá nhân thực hiện

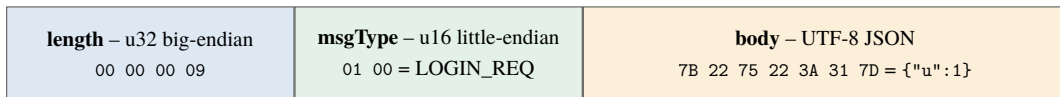
Phân hệ Master Server là **máy chủ TCP thuần .NET 8** — không ASP.NET Core, không SignalR, không WebSocket — chỉ dùng TcpListener và Socket. Trọng tâm là bài toán **đóng khung thông điệp trên luồng byte TCP**. Phạm vi còn gồm thư viện IMasterClient cho client Unity, liên kết game server – master, công cụ kiểm thử tải và hạ tầng triển khai (Docker, systemd, TLS, Terraform).

3.1. Kiến trúc thành phần Master Server



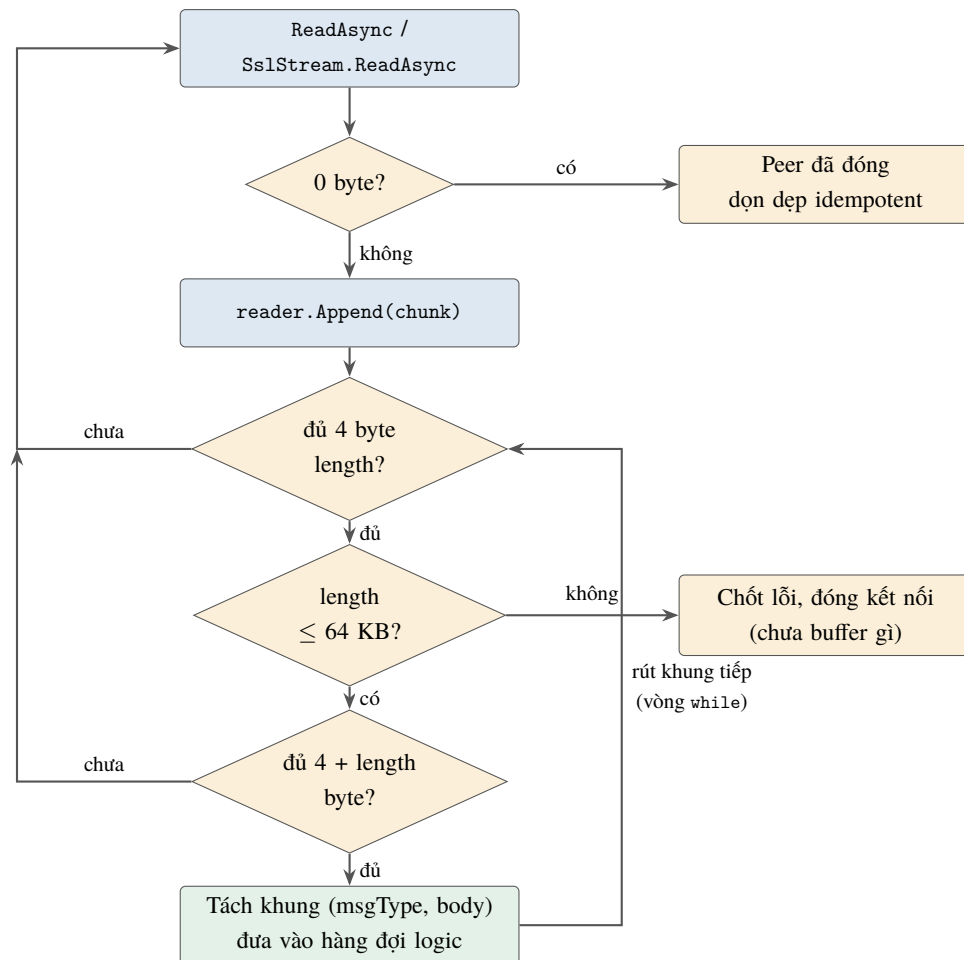
Hình 3.1. Biểu đồ thành phần Master Server: I/O bất đồng bộ trên thread pool, logic trên một luồng duy nhất

3.2. Đóng khung tin trên luồng TCP



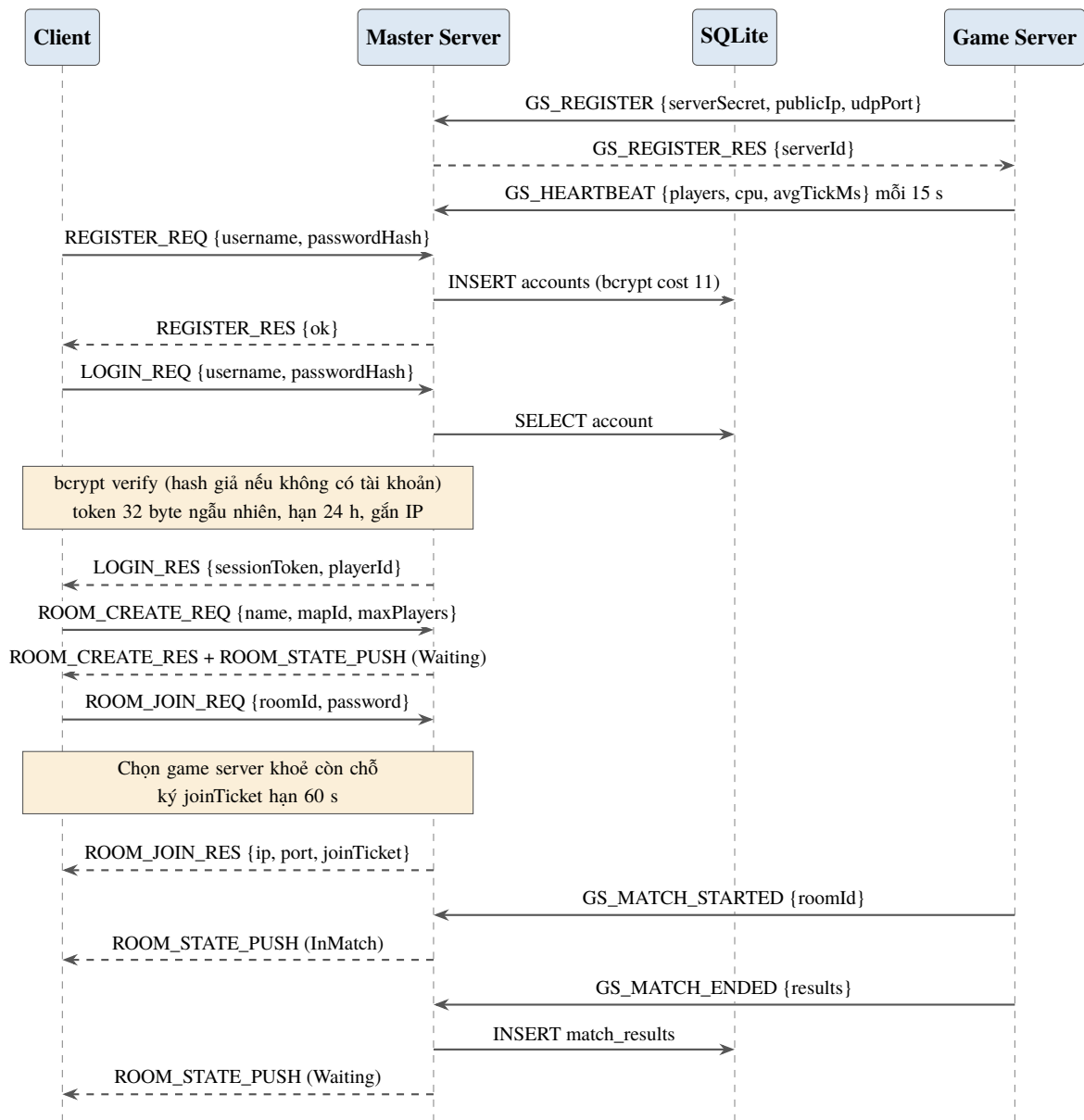
Hình 3.2. Khung tin MSP tham chiếu 13 byte (được ghim trong bộ test conformance)

Trường hợp đọc	Điều xảy ra nếu xử lý sai	Số liệu đo thực tế
Một lần đọc, một khung	Không lỗi – trường hợp ai cũng thử	Chỉ trùng khớp do tình cờ
Dính (glue)	Khung 2..n bị bỏ hoặc kết nối treo	1000 lần Send → 942 lần Receive
Tách (split)	Không bao giờ ghép đủ khung, đọc rác từ tiền tố cắt	1 khung 58 KB → 8 lần Receive
Dính + phần dư đuôi	Cả hai lỗi cùng lúc; phải giữ phần đuôi qua các lần đọc	Kiểm thử riêng



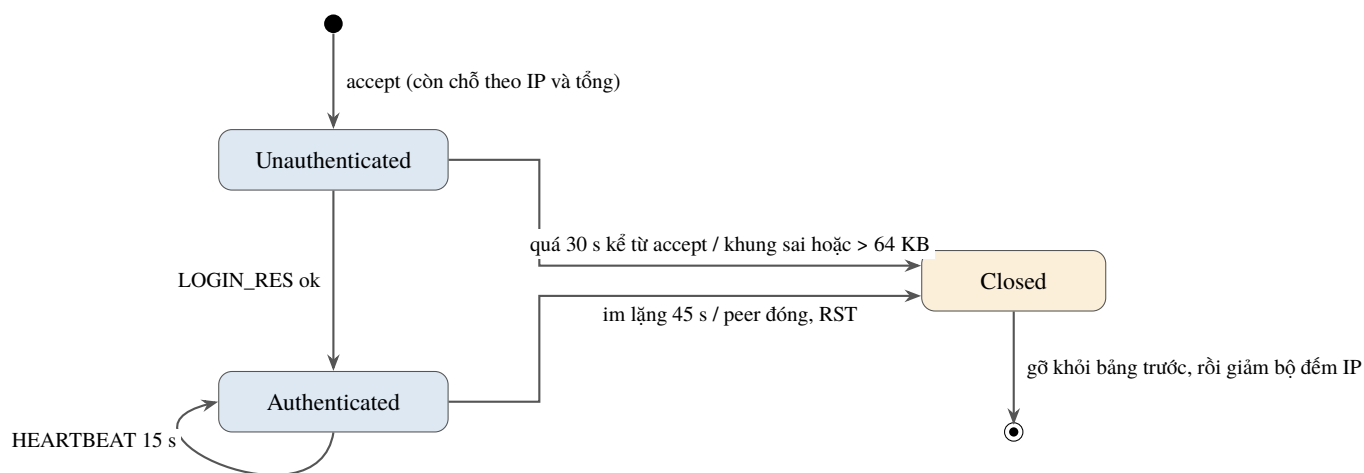
Hình 3.3. Thuật toán bộ đọc tích lũy: rút mọi khung hoàn chỉnh trong vòng lặp while, không phải if

3.3. Luồng dịch vụ sảnh chờ

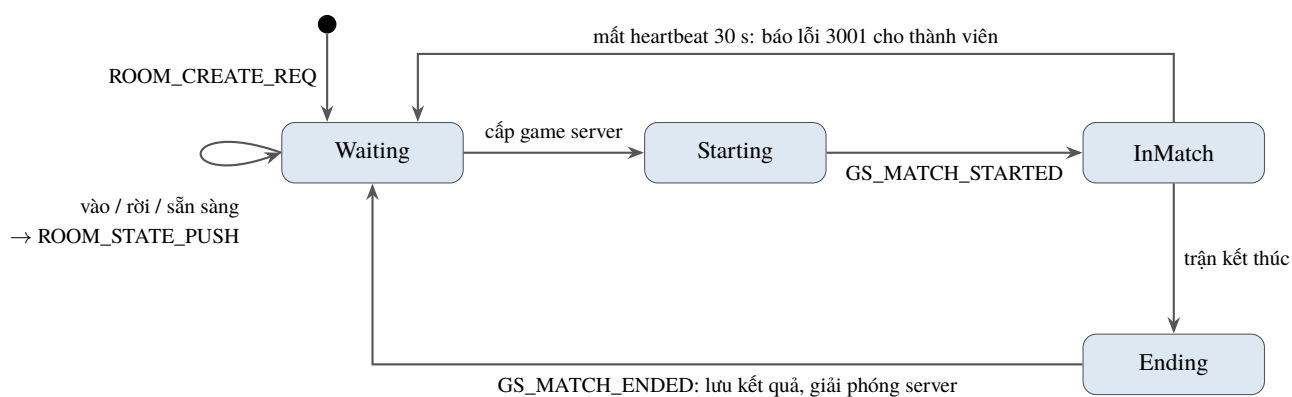


Hình 3.4. Biểu đồ tuần tự dịch vụ sảnh: đăng ký game server, tài khoản, phòng, joinTicket, kết quả trận

3.4. Vòng đời kết nối và phòng



Hình 3.5. Vòng đời một kết nối TCP: hạn tuyệt đối trước xác thực, hạn im lặng sau xác thực



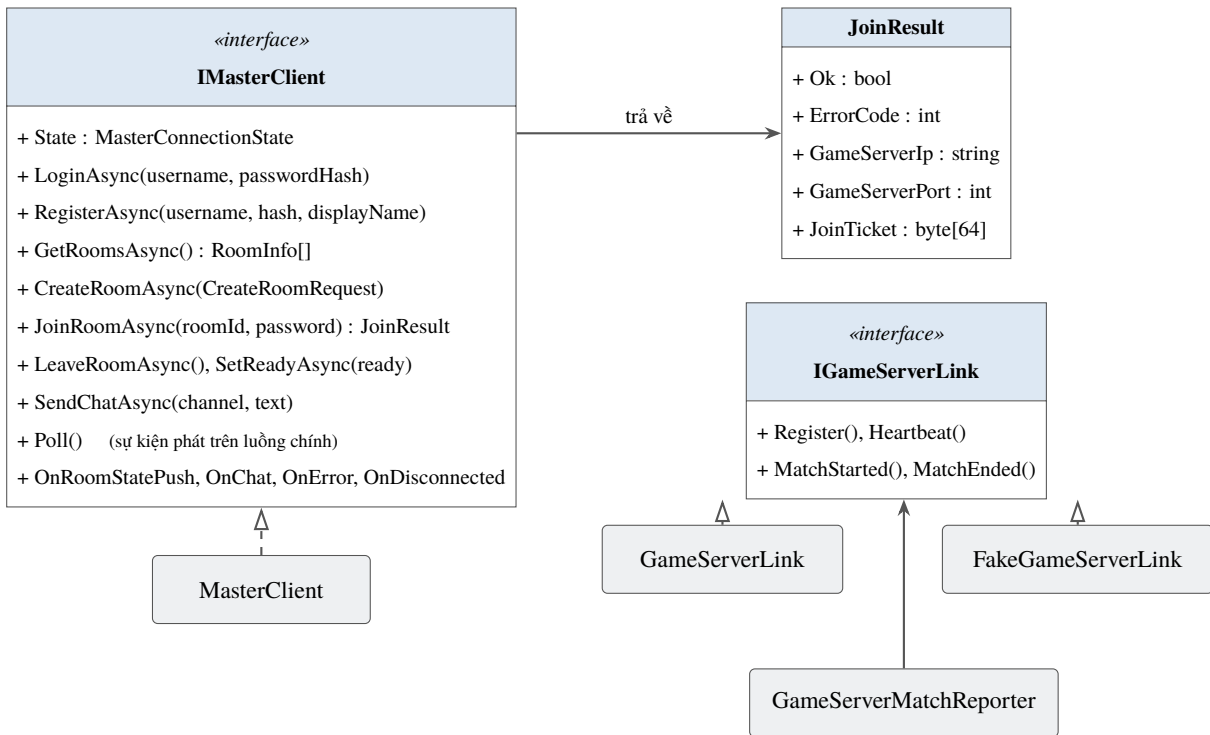
Hình 3.6. Vòng đời phòng (RoomLifecycleState)

3.5. Mô hình dữ liệu

accounts		
INTEGER	player_id	PK
TEXT	username	UNIQUE, NOCASE
TEXT	password_hash	bcrypt
TEXT	display_name	
INTEGER	created_at	
INTEGER	last_login_at	
INTEGER	failed_logins	
INTEGER	locked_until	
INTEGER	is_banned	

Hình 3.7. Lược đồ SQLite của Master Server (phiên và phòng được giữ trong bộ nhớ)

3.6. Giao diện lập trình và giao diện giám sát



Hình 3.8. API cho client Unity và liên kết game server; mọi sự kiện được phát trên luồng chính trong Poll()

```

$ nc 127.0.0.1 27001
{ "uptimeSec": 5412,
  "connections": { "current": 18, "peak": 32,
    "totalAccepted": 1411, "refused": 0,
    "timedOut": 3 },
  "transport": { "tls": true,
    "framesReceived": 88213,
    "tlsHandshakeFailures": 0 },
  "accounts": { "total": 57, "onlineNow": 16 },
  "rooms": { "active": 3, "inMatch": 2,
    "queued": 0 },
  "gameServers": { "registered": 2,
    "healthy": 2, "allocated": 2 },
  "rates": { "loginsPerMin": 4,
    "errorsPerMin": 0 } }

```

```

$ dotnet run --project Ironfront.Tools.LoadTest -- \
  --master 127.0.0.1:27000 --clients 16 \
  --duration 60 --behavior random-walk

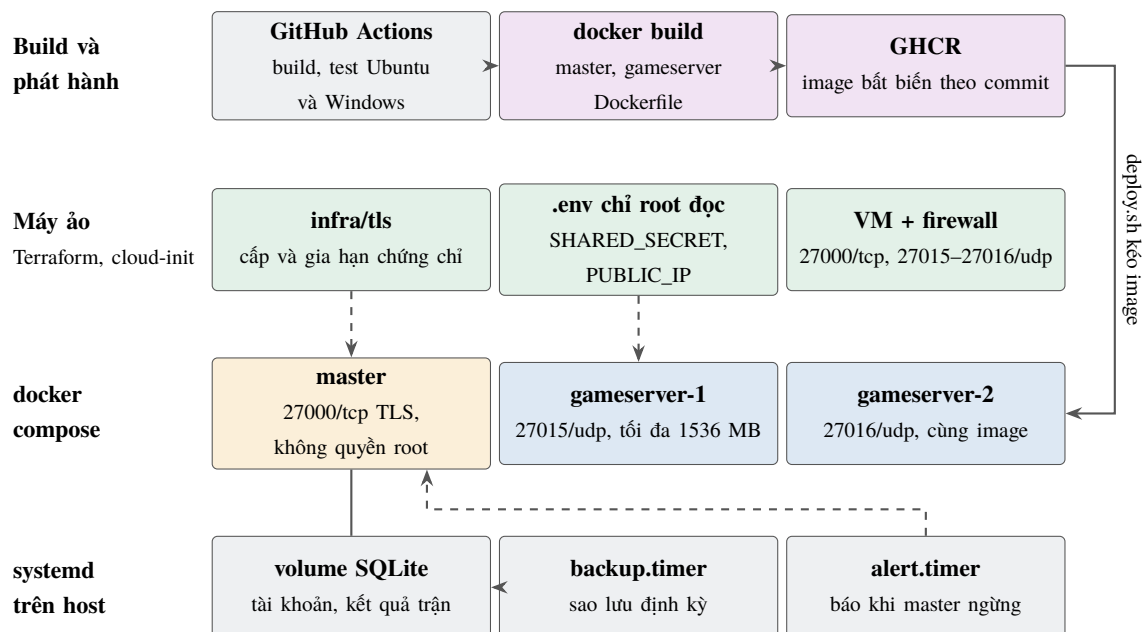
```

scenario	ops	ops/s	p50	p99	fail
random-walk	8636	143.9	0.81ms	5.77ms	0
room-list	366927	12230.0	0.89ms	7.42ms	0
join-leave	3911	130.4	0.72ms	5.68ms	147
32 clients	6208	206.9	1.03ms	8.51ms	0

RAM 64-74 MB; 1000 connections held: 81 MB

Hình 3.9. Giao diện giám sát: điểm cuối metrics JSON trên TCP thô (trái, giá trị minh họa cấu trúc) và báo cáo công cụ kiểm thử tải (phải, số đo thực)

3.7. Hạ tầng triển khai



Hình 3.10. Đường ống triển khai và topo hạ tầng trên VPS (fly.io là phương án thay thế)